

Time as Intervals, Not Instants: A Practical Implementation of Allen’s Interval Algebra Over ISO 8601

Kip Cole ✉

Independent

Abstract

Mainstream programming languages model temporal values as scalar instants: a `Date` is a point, a `DateTime` is a point, and a partial value such as “June 2026” has no canonical representation at all. Yet the temporal values that humans, calendars, and business logic actually manipulate are spans on the timeline — the year 2026, the afternoon of the 15th, an archaeological period, a meeting. `Tempo` is an Elixir library that reverses the polarity: every temporal value is a bounded half-open interval at a declared resolution, set-theoretic combination is closed under that type, and comparison is defined by Allen’s thirteen relations rather than by scalar ordering. The result eliminates an entire class of off-by-one and “end-of-day” bugs by construction, gives partial ISO 8601 values (2026, 2026-06, 2026-06-XX) an unambiguous denotational semantics, and lifts Allen’s algebra from a textbook abstraction to an everyday predicate vocabulary (`overlaps?`, `within?`, `adjacent?`). This extended abstract describes the conceptual model, the unification of partial dates with explicit intervals, and the design of set operations over intervals and interval sets.

2012 ACM Subject Classification Information systems → Temporal data; Software and its engineering → Domain specific languages; Theory of computation → Modal and temporal logics

Keywords and phrases Temporal intervals, Allen’s interval algebra, ISO 8601, calendrical systems, partial dates, set operations on time, domain-specific languages

Digital Object Identifier 10.4230/OASICS...

Related Version Source repository: <https://github.com/kipcole9/tempo>

1 Motivation: the instant–span impedance mismatch

Every mainstream temporal library — Python’s `datetime`, Java’s `java.time`, JavaScript’s `Temporal` proposal, Elixir’s own `Date/Time/DateTime` — shares a design choice that goes largely unexamined: a temporal value is a *scalar instant* on the timeline. A day is the midnight at its start, a month is the midnight on its first, a year is the midnight on January 1. The span that humans actually mean when they say “June 2026” is materialised, if at all, by ad-hoc helpers (`start_of_month`, `end_of_day`) that each project re-implements with its own conventions about inclusivity, time zones, and DST.

This instant-first model has three persistent consequences:

Ambiguity at the type level.

The expression `Date.new(2026, 6, 15)` denotes a 24-hour span in business prose and an instant in the type system. The disagreement is resolved informally, per call site, and is a recurring source of off-by-one bugs at month, year, and DST boundaries. Tiwari et al.’s recent empirical study of 151 date/time bugs in open-source Python projects identifies time-zone-related mistakes as the largest single category, with most bugs traceable to misconceptions about library API conventions and edge-case behaviour [10]; Liva’s earlier work on Java programs reaches a structurally similar conclusion [11].



© Kip Cole;
licensed under Creative Commons License CC-BY 4.0
OpenAccess Series in Informatics

OASICS Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany

42 No representation for partial dates.

43 A user who writes “2022” or “the 1560s” or “approximately June 2026” has produced a
 44 perfectly meaningful temporal value, but the type system has no home for it. Solutions
 45 range from sentinel values (`Date.new(2022, 1, 1)` stands in for “the year”) to parallel type
 46 hierarchies (EDTF [8]) that exist outside the standard library, or to dedicated calendric type
 47 systems such as Bry and Spranger’s CaTTS for Semantic Web query languages [12]. The
 48 fragmentation is structural, not incidental.

49 Comparison is reduced to scalar ordering.

50 Two dates are either $<$, $=$, or $>$. The richer relational structure that Allen identified in 1983 [1]
 51 — *precedes*, *meets*, *overlaps*, *starts*, *during*, *finishes*, *equals*, and their inverses — is invisible at
 52 the type level, and reimplemented case-by-case in business logic that interrogates whether a
 53 meeting “overlaps” the workday or a contract “contains” a billing period.

54 Tempo’s design is the observation that all three problems dissolve simultaneously if
 55 intervals are made the primitive type. There are no points: an apparent “instant” is the
 56 interval of one unit at the finest declared resolution, in the same spirit as the time ontology
 57 $T_{\text{bounded_meeting}}$ of Grüninger and Li [13], in which intervals are the only entities in the
 58 domain and Allen’s relations are derivable from a single primitive (*meets*).

59 Empirical coverage.

60 Against the 151 Python bugs catalogued by Tiwari et al., the design choices described
 61 in the remainder of this paper address every in-scope bug class: 43 (28%) are *avoided by*
 62 *construction* (the naïve / aware datetime split has no Tempo analogue), 56 (37%) are *directly*
 63 *addressed* by explicit machinery (zone propagation through set operations, DST gap rejection
 64 at parse time and IXDTF-offset fold disambiguation per RFC 9557 §4.5, calendar-aware
 65 arithmetic, half-open materialisation, microsecond-precision durations), 28 (19%) are partial,
 66 and 24 (16%) are out of scope (Python deprecations and third-party-library logic such as
 67 holiday tables). *Zero* bugs are classified as architectural gaps. The per-bug classification is
 68 provided as supplementary material at the repository.

69 **2** Intervals as the primitive

70 Every Tempo value is a bounded half-open interval $[first, last)$ at a declared resolution. Inter-
 71 nally an interval is a pair of endpoints, which is the standard finite model of $T_{\text{bounded_meeting}}$
 72 characterised by Grüninger and Li [13] (their Theorem 15): vertices of the directed graph
 73 $\mathfrak{M}^{\text{bounded_meeting}}$ correspond to intervals, edges to the *meets* relation, and structures up to
 74 isomorphism faithfully capture the ontology’s models.

75 A note on terminology before proceeding: *resolution* as used throughout this paper is
 76 a Tempo-side property of an input value (2026 carries year-resolution, 2026-06-15 carries
 77 day-resolution) used for partial-date materialisation, default iteration, and display. It is
 78 *not* the granularity of Bettini, Wang and Jajodia [9] (a lattice of partitions of the time line
 79 that participates in Allen comparisons), and it does *not* appear in $T_{\text{bounded_meeting}}$, where
 80 intervals are individuated solely by their position in the *meets*-structure. Once an interval
 81 is materialised to its $[first, last)$ form, resolution plays no further role in Allen comparisons
 82 or in set-theoretic operations. The word is also distinct from *precision* and *accuracy* in
 83 the measurement-science sense: Tempo makes no claim about the precision of a value’s
 84 construction, only about the extent it denotes.

A second Tempo-side distinction is between *anchored* and *non-anchored* values. An anchored value carries a year component (e.g. 2026-06-15 or 2026); materialising it yields an interval that is an *individual* in $T_{\text{bounded_meeting}}$ in the standard model-theoretic sense — a vertex of $\mathfrak{M}^{\text{bounded_meeting}}$ (Theorem 15) with a definite position in the *meets*-structure. A non-anchored value (a time-of-day template such as T10:30) is not an individual: it is an open expression with the date components as free variables, denoting the family of individuals that result when those variables are bound. Tempo’s `anchor/2` and `align/3 :bound` option supply that binding; the ontological move is *instantiation*, and the result is the intervalised individual the ontology talks about. The two operations Tempo exposes for working with non-anchored values are *projection* (resolving to a specific universal-timeline extent given an anchor) and *selection* (filtering a set of candidate anchors). Neither has a counterpart inside $T_{\text{bounded_meeting}}$ itself, which is silent on schemas; they sit, like resolution, in the engineering layer above.

The library distinguishes two forms of interval that interconvert losslessly.

Implicit spans.

A single ISO 8601 datetime *is* an interval whose span is determined by the next-higher resolution that is not stated:

2026	≡	[2026-01-01, 2027-01-01)
2026-01	≡	[2026-01-01, 2026-02-01)
2026-01-15	≡	[2026-01-15, 2026-01-16)
2026-01-15T10	≡	[2026-01-15T10:00, 2026-01-15T11:00)

The partial-date problem disappears: 2026 is not an under-specified instant, it is a definite interval of one year (the distinction is semantic, not a claim about measurement precision or accuracy). The “last day of the month” problem disappears: the upper bound of 2026-01 is 2026-02-01, computed from the calendar arithmetic of the target calendar (Gregorian, Hebrew, Islamic Civil, ...), and is correct by construction in leap years and across month-length variation.

Explicit spans.

A pair of datetimes written with a range operator — 2026-01-01..2026-02-01 or, in IXDTF-extended form [7], 2026-01-01/2026-02-01 — carries its own boundaries. Its iteration unit is the resolution of its boundaries, not of a partial denotation.

Half-open by design.

The half-open [*first*, *last*) convention is not notational shorthand; it is enforced at every level of the API. Adjacent intervals concatenate cleanly: $[a, b) \cup [b, c) = [a, c)$ with neither overlap nor gap. Iteration yields a sequence whose union is exactly the parent interval. Conversion between implicit and explicit forms is a structural identity, not an arithmetic approximation. Library code that treats the upper bound as inclusive is, by the design contract, a defect. Alternative timestamp representations have been explored at TIME — notably Dyreson and Ahsan’s log-segmented timestamps [14, 15], which decompose a period into power-of-two segments so that standard B⁺-tree indexes can support sequenced and nonsequenced queries on an unmodified DBMS. The two designs occupy different points in the same space: log-segmented timestamps optimise query evaluation in a relational engine; Tempo’s half-open intervals optimise expressiveness and uniformity at the application layer.

3 Allen’s algebra as the predicate vocabulary

Tempo exposes Allen’s thirteen relations [1] directly as the result of its core comparison primitive. The first-order ontology underlying the algebra is characterised by Grüninger and Li [13], who identify the time ontology logically synonymous with Allen’s interval algebra and prove a representation theorem for its models; Tempo is, in effect, an ergonomic software realisation of that ontology over the calendar-aware datetime values defined by ISO 8601. The thirteen relations are returned directly from the relation primitive:

```
Tempo.relation(a, b) ::
  :precedes | :meets | :overlaps | :finished_by |
  :contains | :starts | :equals |
  :started_by | :during | :finishes | :overlapped_by |
  :met_by | :preceded_by
```

Beyond the relations themselves, the library ships Allen’s *composition* table — the transitivity result of [1] Fig. 4 — as a first-class operation, `Tempo.compose/2`: given $A \ r_1 \ B$ and $B \ r_2 \ C$, it returns the set of relations that can hold between A and C . Each of the 169 cells was derived independently, by a difference-bound consistency check over the six endpoints under the half-open convention, and then checked cell-for-cell against the published table. Composition is at present a standalone query rather than the engine of a propagation algorithm — see §6 — but it is the piece on which qualitative reasoning over non-grounded intervals would be built.

This is rarely the right surface for application code, however — business questions tend to name unions of Allen relations. “Does the meeting fall within the workday?” is the set $\{equals, starts, during, finishes\}$; “does this period touch that one?” is the complement of $\{precedes, preceded_by\}$. Hand-rolled inline matching on relation lists is verbose and obscures the intent, so the library promotes the common unions to first-class predicates — `within?/2`, `overlaps?/2`, `adjacent?/2`, `contains?/2` — each defined as a specific subset of Allen’s relations and each readable aloud as the English question it answers.

The predicate vocabulary is not decorative. It is a design discipline: when an example in the cookbook reaches for a relation list or a manual endpoint comparison, that is taken as evidence that the abstraction is missing, and the predicate is added before the example is written. The same discipline applies to durations (`at_least?`, `shorter_than?`) and to interval-set queries (`covered?`, `disjoint?`). The library’s public surface is, deliberately, an English-prose description of Allen’s algebra over ISO 8601 values.

4 Position among temporal formalisms

The choice to make intervals primitive places Tempo on one side of a long-standing divide. Vilain and Kautz [2] take *points* as primitive: an interval is an ordered pair of endpoint points, and an interval relation reduces to a conjunction of point-relations $\{<, =, >\}$ among the four endpoints. The reward is tractability — reasoning in the point algebra is polynomial, whereas constraint reasoning over the full interval algebra is intractable in general [2, 4]. Allen and Hayes [3] take the opposite stance, axiomatising the algebra from the single primitive *meets* and reconstructing points only as derived entities — *moments* (intervals with no proper subinterval) and the meeting-places between intervals. Grüninger and Li’s *T_{bounded_meeting}* [13] continues this interval-only lineage, excluding degenerate intervals and points from the domain entirely; this is the ontology Tempo adopts.

Tempo’s contribution is best read as a synthesis across this divide. *Ontologically* it is interval-only: there are no instants, degenerate intervals are excluded, and the half-open seam realises Hayes’ open-connected model rather than Allen’s closed intervals. *Computationally*, however, comparison projects every endpoint to a real number on a single UTC frame and orders intervals by their endpoints — precisely the Vilain–Kautz reduction, used as an engine beneath an interval ontology. Two consequences follow. First, because Tempo’s intervals are always *fully grounded* (concrete endpoints, never disjunctive qualitative constraints), it never enters the NP-hard regime that motivated the point algebra: relation queries are constant-time endpoint comparisons rather than constraint propagation. Second, the “instant” is recovered not as a primitive point but as the interval of one unit at the value’s *finest declared resolution* — a *moment* in Allen and Hayes’ sense, but relativised to representational precision: a second-resolution datetime is a one-second interval, a microsecond value a one-microsecond interval. This resolution-indexed atomicity, tying interval granularity to ISO 8601 syntax rather than to an absolute atom, is the element of the model without a direct precedent in the formalisms it draws on.

5 Set operations on intervals and interval sets

The current implementation focus is closing the library under set-theoretic operations. The intuition is straightforward — any two intervals can be combined into a (possibly disjoint, possibly zero-member) set of intervals — but the engineering requires careful handling of the implicit/explicit-span distinction, of cross-calendar operands, and of cross-zone arithmetic. A single connected, non-empty result is returned as an `Interval`; every other shape — disjoint, or zero-member — is returned as an `IntervalSet`. An interval is never returned with zero extent, in line with $T_{bounded_meeting}$ ’s exclusion of degenerate intervals from the domain.

Pairwise operations.

`union/2`, `intersection/2`, and `difference/2` on a pair of operands a, b return either an interval or an `IntervalSet`, depending on whether the result is connected. Operands may be supplied in either implicit or explicit form; the library normalises both to the explicit half-open representation, performs the geometric operation, and re-presents the result in the most compact form (a single interval when possible).

n -ary operations and canonicalisation.

Extending the pairwise operations to lists of intervals raises the question of canonical form. Tempo defines the canonical form of an `IntervalSet` as the sorted, coalesced sequence in which no two intervals overlap or meet. Coalescing is the closure of the union operation under iteration: $\bigcup_i [a_i, b_i)$ expressed as a minimal disjoint sequence. This is the form returned by every set-producing operation in the public API. Coalescing also realises the Sum Axiom of $T_{bounded_meeting}$ (axiom 15 in [13]): three chain-meeting intervals $meets(i, j) \wedge meets(j, k) \wedge meets(k, m)$ are summed into the single interval that runs from i ’s start to m ’s end.

Cross-calendar and cross-zone semantics.

Because each operand carries its own calendar and zone, set operations are defined on the underlying UTC reference frame — each endpoint is projected to gregorian seconds (the standard Erlang representation) so that endpoints from different calendars or zones can be ordered in a single linear sequence. The word “instant” is avoided here because Grüninger

and Li’s ontology excludes points from its domain (intervals are the only entities); naming the projection a “UTC instant” would suggest a commitment to a competing point-based ontology. The projection is purely a computational device — a real number used to order endpoints in a single linear frame. The result’s calendar and zone are taken from the first operand, preserving the caller’s intent for downstream display. This first-operand-wins rule is itself a design choice that deserves community scrutiny — alternatives include rejecting heterogeneous operands, or carrying the heterogeneity through into a sum-typed result — and is one of the points the talk would invite feedback on.

Daylight-saving transitions.

The UTC projection interacts with daylight-saving transitions in two distinct ways, both handled at parse time so set operations on the projected values need no further disambiguation. A wall time that falls in a DST *gap* (the non-existent hour at spring-forward) is rejected at construction with a `ZoneGapError` — a stronger guarantee than addressing the bug, because the malformed value cannot be constructed. A wall time that falls in a DST *fold* (the doubled hour at fall-back) is disambiguated by an explicit offset in the IXDTF suffix, per RFC 9557 §4.5: the value `2024-11-03T01:30:00-04:00[America/New_York]` unambiguously picks the pre-fall-back (EDT) period and `2024-11-03T01:30:00-05:00[America/New_York]` picks the post-fall-back (EST) period. The disambiguator is part of the string representation and round-trips through serialisation — a property that distinguishes this design from meta-attribute schemes (such as Python’s `fold` parameter) in which the choice is carried out-of-band and lost on serialisation.

Beyond the primitives.

Once `union`, `intersection`, and `difference` are closed, the natural extensions follow: symmetric difference, containment queries, the inverse of an interval set against a bounding context (“free time” given a calendar of meetings), and reductions across an `IntervalSet` (total duration, longest gap, densest hour).

6 Discussion and open questions

Tempo is a working library; this submission’s contribution is a *design* contribution rather than a *result* contribution, and the talk would aim to make the design choices legible to an audience whose research is the theory the library implements. Three questions seem particularly worth the TIME audience’s attention:

1. **Granularity and resolution.** Tempo’s next-higher-undefined-resolution rule for implicit spans is operationally simple but does not reference the more general treatment of temporal granularity in the database literature [9] nor the calendric type constraints in CaTTS [12]. Is the simple rule a sufficient model in practice, or does it accumulate edge cases as the calendar set grows (lunar, sidereal, fiscal)?
2. **Allen’s algebra over interval sets.** The thirteen relations, and the $T_{\text{bounded_meeting}}$ ontology that underlies them, are defined on convex intervals only [13]. Their lifting to interval sets admits several reasonable definitions (existential, universal, set-of-relations). The library now offers two of the three — the existential lifting through the predicates, and the set-of-relations lifting through `relation_matrix/2`, which returns the relation holding between every pair of members — but which should be the *default* is not obvious, and a principled extension of the ontology to non-convex interval sets would settle it.

254 **3. Cross-calendar comparison.** Comparing a Hebrew date with a Gregorian one resolves
 255 cleanly when both project to the same proleptic UTC timeline, but the timeline is
 256 itself a modelling choice (TAI? UT1? a calendar-specific civil time?). Tempo’s current
 257 choice is pragmatic; a principled framework would be a worthwhile contribution from the
 258 symposium back to the library.

259 The wider goal is to demonstrate, by existence proof, that the theoretical structure of
 260 temporal reasoning translates into an ordinary application library — one that an engineer
 261 writes `Tempo.overlaps?(a, b)` into without thinking of it as a research artefact. The same
 262 primitive reaches beyond set algebra: a thin constraint layer reduces a network of partially
 263 known intervals to a Simple Temporal Problem [16], reproducing the chronological-network
 264 model of Levy et al. [17] — realised in the ChronoLog tool [18] — as a library feature rather
 265 than a bespoke application.¹ The talk would walk through the design, the choices made, the
 266 choices still open, and invite the symposium to sharpen them.

267 — References —

- 268 1 James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the*
 269 *ACM*, 26(11):832–843, 1983. doi:10.1145/182.358434
- 270 2 Marc B. Vilain and Henry A. Kautz. Constraint propagation algorithms for temporal reasoning.
 271 In *Proceedings of the 5th National Conference on Artificial Intelligence (AAAI-86)*, pages
 272 377–382, 1986.
- 273 3 James F. Allen and Patrick J. Hayes. Moments and points in an interval-based temporal logic.
 274 *Computational Intelligence*, 5(3):225–238, 1989. doi:10.1111/j.1467-8640.1989.tb00329.x
- 275 4 Bernhard Nebel and Hans-Jürgen Bürckert. Reasoning about temporal relations: A maxi-
 276 mal tractable subclass of Allen’s interval algebra. *Journal of the ACM*, 42(1):43–66, 1995.
 277 doi:10.1145/200836.200848
- 278 5 International Organization for Standardization. *ISO 8601-1:2019 — Date and time — Repre-*
 279 *sentations for information interchange — Part 1: Basic rules*, 2019.
- 280 6 International Organization for Standardization. *ISO 8601-2:2019 — Date and time — Repre-*
 281 *sentations for information interchange — Part 2: Extensions*, 2019.
- 282 7 U. Sharma and C. Bormann, editors. *Date and Time on the Internet: IXDTF Extensions*.
 283 Internet-Draft draft-ietf-sedate-datetime-extended-09, IETF, 2024. [https://www.ietf.org/](https://www.ietf.org/archive/id/draft-ietf-sedate-datetime-extended-09.html)
 284 [archive/id/draft-ietf-sedate-datetime-extended-09.html](https://www.ietf.org/archive/id/draft-ietf-sedate-datetime-extended-09.html)
- 285 8 Library of Congress. *Extended Date/Time Format (EDTF) Specification*, 2019. <https://www.loc.gov/standards/datetime/>
- 286 9 Claudio Bettini, Sushil Jajodia, and Sean X. Wang. *Time Granularities in Databases, Data*
 287 *Mining, and Temporal Reasoning*. Springer, 2000.
- 288 10 Shrey Tiwari, Serena Chen, Alexander Joukov, Peter Vandervelde, Ao Li, and Rohan Padhye.
 289 It’s about time: An empirical study of date and time bugs in open-source Python software.
 290 In *Proceedings of the 22nd International Conference on Mining Software Repositories (MSR*
 291 *’25)*. IEEE/ACM, 2025. Distinguished Paper Award. [https://rohan.padhye.org/files/](https://rohan.padhye.org/files/datetimebugs-msr25.pdf)
 292 [datetimebugs-msr25.pdf](https://rohan.padhye.org/files/datetimebugs-msr25.pdf)
- 293 11 Giovanni Liva. *Modeling Time in Java Programs for Automatic Error Detection*. PhD thesis,
 294 Alpen-Adria-Universität Klagenfurt, 2018.
- 295 12 François Bry and Stephanie Spranger. Towards a multi-calendar temporal type system for
 296 (Semantic) Web query languages. In *Principles and Practice of Semantic Web Reasoning*
 297 *(PPSWR 2004)*, volume 3208 of *Lecture Notes in Computer Science*, pages 90–104. Springer,
 298 2004. doi:10.1007/978-3-540-30122-6_8
- 299

¹ Developed in a companion paper, *Chronological Networks as a Library Primitive: ChronoLog over an Interval-Native Date/Time Stack*.

- 300 **13** Michael Grüninger and Zhuojun Li. The time ontology of Allen’s interval algebra. In *24th*
301 *International Symposium on Temporal Representation and Reasoning (TIME 2017)*, volume
302 90 of *LIPIcs*, pages 16:1–16:16. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2017.
303 doi:10.4230/LIPIcs.TIME.2017.16
- 304 **14** Curtis E. Dyreson and M. A. Manazir Ahsan. Achieving a sequenced, relational query language
305 with log-segmented timestamps. In *28th International Symposium on Temporal Representation*
306 *and Reasoning (TIME 2021)*, volume 206 of *LIPIcs*, pages 14:1–14:13. Schloss Dagstuhl –
307 Leibniz-Zentrum für Informatik, 2021. doi:10.4230/LIPIcs.TIME.2021.14
- 308 **15** Curtis E. Dyreson. Optimization of nonsequenced queries using log-segmented timestamps.
309 In *30th International Symposium on Temporal Representation and Reasoning (TIME 2023)*,
310 volume 278 of *LIPIcs*, pages 13:1–13:15. Schloss Dagstuhl – Leibniz-Zentrum für Informatik,
311 2023. doi:10.4230/LIPIcs.TIME.2023.13
- 312 **16** Rina Dechter, Itay Meiri, and Judea Pearl. Temporal constraint networks. *Artificial Intelligence*,
313 49(1–3):61–95, 1991. doi:10.1016/0004-3702(91)90006-6
- 314 **17** Eythan Levy, Gilles Geeraerts, Frédéric Pluquet, Eli Piasetzky, and Alexander Fantalkin.
315 Chronological networks in archaeology: A formalised scheme. *Journal of Archaeological Science*,
316 124:105225, 2020. doi:10.1016/j.jas.2020.105225
- 317 **18** ChronoLog. <http://chrono.ulb.be>.